

Final Exam Solution (12.19, Friday)

Fall, 2025

(Python Basics)

Answer #1

```
(3, 'hello') 1
```

Answer #2

```
[4, 9]
```

Answer #3

```
no water
cat drinks water
```

Answer #4

```
[5, 6, 7]
5
6
```

Answer #5

```
call
forward
done
Result:  18
```

(Understanding PyTorch)

Answer #6

```
torch.Size([3, 2])
torch.Size([2, 2, 3])
```

Answer #7

```
torch.Size([1, 3])
torch.Size([3, 1])
```

Answer #8

```
torch.Size([4, 2])  
torch.Size([2, 3])  
torch.Size([2])
```

(Understanding CNN)**Answer #9**

For a 2D convolution, the spatial output size (height and width) is

$$\frac{64 + 2 \cdot 1 - 4}{2} + 1 = 32$$

So the output height and width are **32 x 32**.

(Understanding NLP)**Answer #10**

```
['Hello', 'world']  
[' ', ' ', '!']
```

Answer #11

(a) The training objective of CBOW is to predict the center word given its surrounding context words.

(b) Negative Sampling optimizes the model by transforming the expensive full Softmax into a binary classification task. In CBOW, it treats the true (context, center) word pair as a positive example and a small number of randomly sampled unrelated words as negative examples. The model is trained to increase similarity for positive pairs and decrease similarity for negative ones.

Answer #12

Without loss of generality, we can consider the 2-dimensional subspace spanned by the vectors **a** and **b**. By choosing a coordinate system where **a** aligns with the positive *x*-axis, the vectors can be represented in this plane as:

$$\mathbf{a} = (1, 0) \quad \text{and} \quad \mathbf{b} = (\cos \theta, \sin \theta).$$

Using the definition of the dot product in Euclidean space:

$$\mathbf{a} \cdot \mathbf{b} = (1)(\cos \theta) + (0)(\sin \theta) = \cos \theta.$$

(Understanding the Transformer)**Answer #13**

(a) Scaled scores can be calculated as

$$QK_1^\top = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = 2,$$
$$QK_2^\top = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 0.$$

Since $d_k = 2$, we divide by $\sqrt{2}$:

$$s_1 = \frac{2}{\sqrt{2}} = \sqrt{2},$$
$$s_2 = \frac{0}{\sqrt{2}} = 0.$$

(b) The weights are

$$\alpha_1 = \frac{e^{\sqrt{2}}}{e^{\sqrt{2}} + e^0} = \frac{e^{\sqrt{2}}}{e^{\sqrt{2}} + 1}, \quad \alpha_2 = \frac{e^0}{e^{\sqrt{2}} + e^0} = \frac{1}{e^{\sqrt{2}} + 1}.$$

Clearly, $\alpha_1 + \alpha_2 = 1$.

(c) Compute the weighted sum:

$$Y = \alpha_1 V_1 + \alpha_2 V_2 = \alpha_1 \begin{bmatrix} 1 & 2 \end{bmatrix} + \alpha_2 \begin{bmatrix} 3 & 0 \end{bmatrix}.$$

Thus,

$$Y = \begin{bmatrix} \alpha_1 \cdot 1 + \alpha_2 \cdot 3 & \alpha_1 \cdot 2 + \alpha_2 \cdot 0 \end{bmatrix} = \begin{bmatrix} \alpha_1 + 3\alpha_2 & 2\alpha_1 \end{bmatrix}.$$

Substitute $\alpha_1 = \frac{e^{\sqrt{2}}}{e^{\sqrt{2}}+1}$ and $\alpha_2 = \frac{1}{e^{\sqrt{2}}+1}$:

$$Y = \begin{bmatrix} \frac{e^{\sqrt{2}}}{e^{\sqrt{2}}+1} + \frac{3}{e^{\sqrt{2}}+1} & \frac{2e^{\sqrt{2}}}{e^{\sqrt{2}}+1} \end{bmatrix} = \begin{bmatrix} \frac{e^{\sqrt{2}}+3}{e^{\sqrt{2}}+1} & \frac{2e^{\sqrt{2}}}{e^{\sqrt{2}}+1} \end{bmatrix}.$$

Answer #14

- In the Transformer, **positional encodings (PE)** are added element-wise to the input word embeddings before they are fed into the first Transformer block. Mathematically, for a token at position pos , the final input vector x_{pos} is:

$$x_{pos} = \text{Embedding}(w_{pos}) + \text{PE}(pos)$$

- **Positional Encodings (PE)** can be computed using sinusoidal functions with different frequencies.
- For any fixed offset k , PE_{pos+k} can be expressed as a linear function of PE_{pos} . This property allows the model to capture relative positional relationships between tokens (such as distances between words) independently of their absolute positions, enabling better generalization to sequences longer than those seen during training.

(Understanding Diffusion Models)

Answer #15

(a) The core idea of diffusion models is to generate data by learning to reverse a gradual noise-adding process. They transform simple random noise (such as Gaussian noise) into complex data samples through an iterative denoising procedure.

- The **forward process** (diffusion process) gradually destroys the structure of the data by adding small amounts of Gaussian noise to the original data sample x_0 over T steps, eventually producing pure noise $x_T \sim \mathcal{N}(0, I)$. This process is fixed and does not involve learning.
- The **reverse process** (denoising process) aims to recover the data structure. A neural network is trained to predict and remove the noise at each step. Starting from random noise x_T , the model iteratively applies the learned denoising steps to generate a new data sample x_0 .

(b) Diffusion models have several advantages compared to GANs and VAEs.

- Diffusion models are more stable to train than GANs. GAN training often suffers from issues such as mode collapse and unstable adversarial optimization, whereas diffusion models are trained using a stable objective and do not require adversarial training.
- Diffusion models typically produce higher-quality and more diverse samples. By gradually refining samples through an iterative denoising process, they are able to capture complex data distributions more accurately than VAEs, which often generate blurry samples due to their strong distributional assumptions.

Answer #15

Without transposition, $\mathbf{q}$ and $\mathbf{k}$ (for one batch) are both $L \times D$ matrices. But the product

$$(L \times D)(L \times D)$$

is not a valid matrix multiplication because the inner dimensions do not match.

To perform a valid matrix multiplication of the form

$$(L \times D)(D \times L),$$

we must transpose the key matrix as follows:

$$\mathbf{k}^\top \in \mathbb{R}^{D \times L}.$$

Since the key matrix, $\mathbf{k}$ has shape (B, L, D) , the following PyTorch code

```
k.transpose(-2, -1)
```

swaps the last two dimensions, producing shape (B, D, L) . Here `transpose(-2, -1)` means “swap the last two dimensions”.

Therefore, we obtain

$$(B, L, D) \cdot (B, D, L) \rightarrow (B, L, L),$$

which is the desired batched attention score tensor.